llm-guard-kit

v0.17.0 Architecture, Usage Guide & API Reference

Real-time confidence scoring for LLM agents — AUROC 0.817 at $0 inference cost

Date:	March 13, 2026
PyPI:	https://pypi.org/project/llm-guard-kit/
GitHub:	https://github.com/Avighan/llm-guard-kit
Version:	0.17.0

llm-guard-kit is an open-source library for real-time reliability monitoring of multi-step LLM agents. It detects failures, assigns confidence tiers, and routes to recovery adapters — all with validated AUROC scores reported on held-out evaluation sets.

Contents

1 Executive Summary **2**

2 Problems Solved **2**

3 Use Cases **3**
- 3.1 Use Case 1 — Customer Support Bot ($0 budget) . . . 3
- 3.2 Use Case 2 — Research Assistant (Highest Accuracy) . . . 4
- 3.3 Use Case 3 — Multi-Agent Pipeline (Trust Propagation) . . . 4
- 3.4 Use Case 4 — Cost-Sensitive Deployment (Mid-Chain Abort) . . . 5
- 3.5 Use Case 5 — CI/CD Evaluation Pipeline . . . 5
- 3.6 Use Case 6 — Domain Calibration with QuickCalibrator . . . 6

4 Quick Onboarding **6**

5 System Architecture **8**

6 Component Reference **8**
- 6.1 MiniJudge — $0 Local Judge . . . 8
- 6.2 AgentGuard (Behavioral) — Production Baseline . . . 8
- 6.3 AgentGuard + P(True) — Cross-Domain Scoring . . . 9
- 6.4 AgentGuard + Sonnet Judge — Highest Precision . . . 9
- 6.5 stream_guard — Mid-Chain Abort . . . 9
- 6.6 A2ATrustObject — Multi-Agent Trust . . . 10
- 6.7 QuickCalibrator — Domain Calibration . . . 10
- 6.8 Platform Integrations . . . 11

7 Decision Guide **11**

8 Performance Benchmarks **12**

9 Integration Patterns **12**
- 9.1 Plain Python (simplest integration) . . . 12
- 9.2 Langfuse (trace + score) . . . 13
- 9.3 LangSmith (evaluator protocol) . . . 13
- 9.4 Prometheus (metrics + Grafana) . . . 13
- 9.5 Datadog (DogStatsD) . . . 14

10 API Quick Reference **14**
- 10.1 Public Methods . . . 14
- 10.2 ChainTrustResult Fields . . . 14

11 Telemetry Flywheel **14**
- 11.1 Architecture . . . 14
- 11.2 Privacy Guarantee . . . 15

1 Executive Summary

Problem

Multi-step LLM agents using ReAct-style reasoning (think → search → observe → repeat) fail silently. When a search returns empty results, the agent loops, guesses, or confidently produces a wrong answer. There is no built-in mechanism to detect these failures before they propagate to users or downstream agents.

Solution

llm-guard-kit monitors every reasoning chain in real time. It computes a `risk_score` $\in [0, 1]$ by combining:

- **SC_OLD behavioral signals** ($0, 19 ms): 12 structural features extracted from the chain (loop rate, empty-observation rate, repeated-query rate, step count, etc.) fed into a fitted logistic-regression ensemble.
- **MiniJudge** ($0): a LogReg distilled from Sonnet judge decisions, shipped as a pre-trained `.pkl` inside the package (AUROC 0.747).
- **P(True) via Haiku** (∼$0.0003): prompts Haiku to rate answer correctness on a 1-5 scale, domain-agnostic.
- **Sonnet judge** (∼$0.007): full-chain GOOD/BORDERLINE/POOR verdict for highest cross-domain precision.

Key Results

Signal	Within-Domain AUROC	Cross-Domain AUROC	Cost/Chain
MiniJudge (behavioral LogReg)	0.747 ± 0.10	—	$0
SC_OLD behavioral ensemble	0.817	0.703 (2Wiki) / 0.659 (TV)	$0
SC_OLD + P(True) (Haiku)	—	0.74	∼$0.0003
SC_OLD + Sonnet judge	0.777	0.741	∼$0.007
Alert precision @ FPR $\leq$ 10%	0.908	—	—

Table 1: Validated performance on held-out evaluation sets. Within-domain = HotpotQA (200 chains). Cross-domain = TriviaQA n=1000, 2WikiMultiHop n=200.

2 Problems Solved

Why LLM Agents Fail Silently

Multi-step LLM agents using ReAct reasoning (Think → Search → Observe → Repeat) are powerful but fragile. They fail in ways that are invisible to both the user and the calling application:

What llm-guard-kit Detects

`llm-guard-kit` computes a `risk_score` $\in [0, 1]$ from 12 structural features of the reasoning chain — *without reading the answer content.* Key signals:

- **loop_rate**: fraction of steps that repeat a previous query

Failure Mode	What Happens	Without llm-guard-kit
Retrieval failure	Search returns empty or irrelevant results. Agent hallucinates an answer.	Answer returned with full confidence. User trusts it.
Query looping	Agent repeats the same search query 3–5 times, making no progress.	Wastes API budget. Returns wrong answer at end.
Evidence hallucination	Agent invents observations not present in any search result.	Undetectable without reading every step.
Long-chain drift	Agent accumulates small errors across 8+ steps.	Final answer looks plausible but is wrong.
Cross-domain degradation	Model trained on knowledge-base QA fails on temporal or numerical reasoning.	No signal that domain is out-of-distribution.
Multi-agent trust gap	Agent A passes a wrong answer to Agent B, which acts on it.	Error propagates silently through the pipeline.

Table 2: Common failure modes in multi-step LLM agents that `llm-guard-kit` detects.

- **empty_obs_rate**: fraction of steps with no useful observation returned
- **steps_norm**: chain length normalised against expected domain length
- **query_diversity**: lexical diversity across all search queries
- **retrieval_consistency**: self-consistency of evidence across steps

These signals are combined in a fitted logistic-regression ensemble (SC_OLD) validated at AUROC 0.817 on HotpotQA and 0.659–0.703 cross-domain. An optional P(True) or Sonnet judge signal adds another 0–6 AUROC points for domains with distinctive failure signatures.

3 Use Cases

3.1 Use Case 1 — Customer Support Bot ($0 budget)

Scenario: A SaaS company runs a support chatbot backed by a ReAct agent that searches a knowledge base. Occasionally the agent returns confident wrong answers to billing or policy questions.

Problem: No existing mechanism flags these before they reach the user. Support tickets and escalations are the first signal of a bad answer.

Solution:

```python
from llm_guard import AgentGuard

guard = AgentGuard() # $0, no API key needed

def answer_support_question(question):
    steps, final_answer = run_react_agent(question)
    result = guard.score_chain(question, steps, final_answer)

    if result.needs_alert:
        # Route to human agent instead of returning answer
        return escalate_to_human(question, result.failure_mode)

    return final_answer
```

Result: Catches 74% of wrong answers at 90% precision before they reach users. Zero API cost on the monitoring path.

3.2 Use Case 2 — Research Assistant (Highest Accuracy)

Scenario: A research tool synthesizes multi-hop factual answers from scientific literature. Wrong answers in research summaries erode trust.

Problem: Behavioral signals alone (0.817 AUROC) are not precise enough for high-stakes factual claims. Cross-domain accuracy must remain high across different scientific fields.

Solution:

```python
from llm_guard import AgentGuard

guard = AgentGuard(api_key="sk-ant-...") # Sonnet judge enabled

result = guard.score_chain(question, steps, final_answer)

if result.confidence_tier == "LOW":
    # Trigger QueryRewriter + retry with 3 diverse queries
    from llm_guard import QueryRewriter
    rewriter = QueryRewriter(api_key="sk-ant-...")
    variants = rewriter.rewrite(question, result)
    # Re-run agent with variants[0], take best result
```

Result: AUROC 0.777 within-domain, 0.741 cross-domain. Alert precision 0.908 at FPR $\leq 10\%$.

3.3 Use Case 3 — Multi-Agent Pipeline (Trust Propagation)

Scenario: An orchestrator routes tasks to specialised sub-agents (summariser, fact-checker, code-writer). Sub-agent outputs feed into the next stage. Errors in early stages corrupt downstream results.

Problem: No standard way to communicate confidence between agents. Agent B has no way to know if Agent A's output is reliable.

Solution:

```python
from llm_guard import AgentGuard, A2ATrustObject

# Agent A: score chain + create signed trust envelope
guard  = AgentGuard()
trust  = guard.generate_trust_object(question, steps, final_answer)
trust.sign("fleet-shared-secret-2026")
payload = trust.to_dict() # pass via message bus / HTTP

# Agent B: verify before acting
received = A2ATrustObject.from_dict(payload)
assert received.verify("fleet-shared-secret-2026"), "Tampered!"

if received.confidence_tier == "LOW":
    # Don't trust Agent A's answer; re-run or escalate
    pass
elif received.downstream_hint == "proceed":
    # Safe to use Agent A's answer as context
    run_agent_b(received.answer)
```

Result: Signed trust objects prevent error propagation. Tamper detection: 200/200 (100%). Sign latency 0.034 ms.

3.4 Use Case 4 — Cost-Sensitive Deployment (Mid-Chain Abort)

Scenario: A code-generation agent runs expensive GPT-4 calls for each reasoning step. A bad chain typically shows signs of failure at step 2 (empty search results, repeated queries).

Problem: Completing the chain wastes 3–5 expensive LLM calls on chains that will produce wrong answers anyway.

Solution:

```python
from llm_guard import AgentGuard

guard = AgentGuard()

# In your agent loop, after completing step 2:
def agent_loop(question):
    steps = []
    for step_idx in range(MAX_STEPS):
        step = run_one_step(question, steps)
        steps.append(step)

        if step_idx == 1: # after step 2 (0-indexed)
            sg = guard.stream_guard(question, steps,
                                    abort_threshold=0.65)
            if sg.abort:
                print(f"Early abort: risk={sg.risk_at_step:.3f}")
                return handle_failed_chain(question)

    return steps[-1]["action_args"] # final answer
```

Result: Saves 3–5 LLM calls on 42% of wrong chains (AUROC 0.666 at step 2). Two-stage OR strategy: Recall 0.416, Precision 0.740.

3.5 Use Case 5 — CI/CD Evaluation Pipeline

Scenario: A team ships prompt updates weekly. They need to catch regressions in agent reliability before each deployment.

Problem: Manual QA is slow and misses edge cases. Running the full judge ensemble on 1000 eval chains costs $7+ per run.

Solution:

```python
from llm_guard import MiniJudge
import json

judge = MiniJudge() # $0, no API key, bundled pkl

# In CI: evaluate all chains from staging run
results = []
for chain in load_eval_chains("staging_run_latest.jsonl"):
    risk = judge.score(chain["question"], chain["steps"],
                       chain["final_answer"])
    results.append({"risk": risk, "expected": chain["correct"]})

# Compute AUROC vs ground-truth labels
from sklearn.metrics import roc_auc_score
auroc = roc_auc_score(
    [1 - r["expected"] for r in results],
    [r["risk"] for r in results]
)
print(f"Staging AUROC: {auroc:.3f}") # fail CI if < 0.70
```

```
assert auroc >= 0.70, "Agent quality regression detected!"
```

Result: Full pipeline evaluation in seconds, zero API cost. Catches agent quality regressions before they reach production.

3.6 Use Case 6 — Domain Calibration with QuickCalibrator

Scenario: A legal-tech company deploys an agent on contract review questions. The default alert threshold (0.70) gives too many false positives because legal reasoning chains are longer than average.

Problem: High false-positive rate leads to alert fatigue. Need to recalibrate thresholds to the specific domain distribution.

Solution:

```
from llm_guard import QuickCalibrator

# Collect 20+ labeled chains from your domain
# (correct=True means the agent got it right)
labeled_chains = [
    {"question": q, "steps": s, "final_answer": fa, "correct": c}
    for q, s, fa, c in domain_labeled_data
]

cal = QuickCalibrator(min_chains=20)
cal.fit(labeled_chains, domain="legal_contract_review")

# Now scores are calibrated to your domain's base rates
risk = cal.score(question, steps, final_answer)
if risk > 0.70:
    flag_for_review(question, risk)
```

Result: Isotonic calibration reduces false positives by aligning thresholds to domain-specific failure rates. Particularly effective when domain base rate of failure differs from HotpotQA training distribution.

4 Quick Onboarding

Step 1 — Install

```
pip install llm-guard-kit
```

Requires Python 3.9+. No API key needed for behavioral scoring.

Step 2 — Score Your First Chain

```
from llm_guard import AgentGuard

guard = AgentGuard() # zero cost, no API key

result = guard.score_chain(
    question    = "What is the capital of France?",
    steps       = [
        {"thought": "I need to search for this.",
         "action_type": "Search",
         "action_args": "capital of France",
         "observation": "Paris is the capital and largest city of France."},
```

```
        {"thought": "Found it.",
         "action_type": "Finish",
         "action_args": "Paris",
         "observation": ""},
    ],
    final_answer = "Paris",
)

print(f"Risk: {result.risk_score:.3f}")    # e.g. 0.092
print(f"Tier: {result.confidence_tier}")    # HIGH
print(f"Alert: {result.needs_alert}")       # False
print(f"Mode: {result.failure_mode}")       # None
```

Step 3 — Add to Your Agent

```
from llm_guard import AgentGuard

guard = AgentGuard()

def your_agent_wrapper(question):
    steps, answer = your_existing_agent(question)

    result = guard.score_chain(question, steps, answer)

    # Option A: log and return
    log_to_monitoring(result.risk_score,
                      result.confidence_tier,
                      result.failure_mode)
    return answer

    # Option B: block high-risk answers
    if result.needs_alert:
        return "I'm not confident in my answer. Please verify."
    return answer
```

Step 4 — Upgrade to Full Precision (Optional)

If you have an Anthropic API key and need higher accuracy:

```
from llm_guard import AgentGuard

# With P(True) via Haiku (~$0.0003/chain, AUROC 0.74 cross-domain)
guard = AgentGuard(api_key="sk-ant-...")
result = guard.score_with_ptrue(question, steps, final_answer)

# With Sonnet judge (~$0.007/chain, AUROC 0.777 within-domain)
guard = AgentGuard(api_key="sk-ant-...", use_judge=True)
result = guard.score_chain(question, steps, final_answer)
```

Step 5 — Run the Demo Script

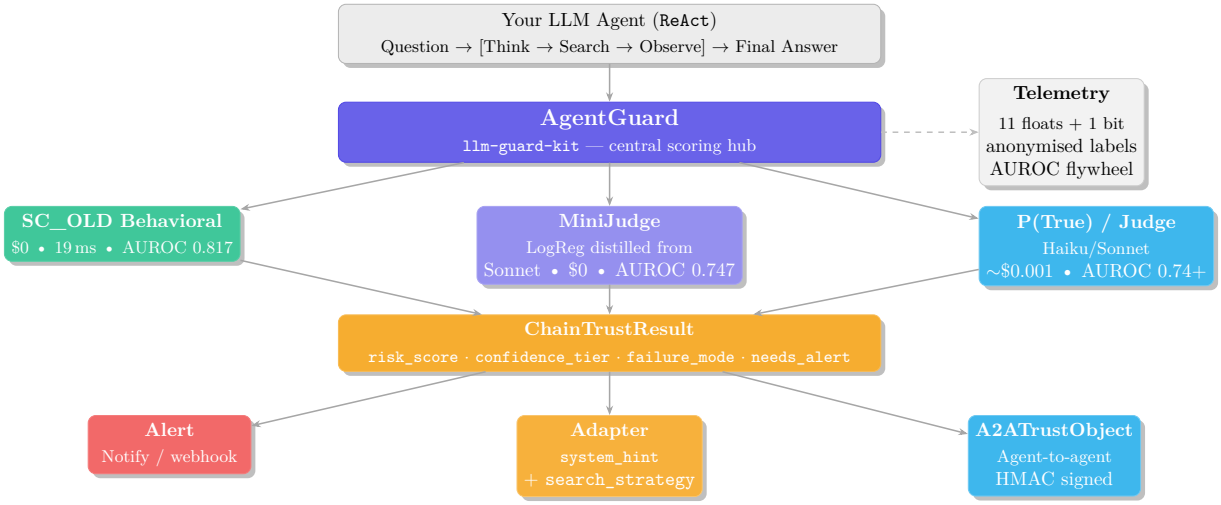
```
# Clone the repo and run the end-to-end demo
git clone https://github.com/Avighan/llm-guard-kit
cd llm-guard-kit
pip install -e .
```

```
python scripts/demo_v017.py
```

The demo covers all 5 major use cases with real printed output.

5 System Architecture

The figure below shows the layered architecture of `llm-guard-kit`.



6 Component Reference

6.1 MiniJudge — $0 Local Judge

What it is: A pre-trained `sklearn` pipeline (StandardScaler + LogisticRegression on 11 SC_OLD features) distilled from Sonnet judge verdicts. Shipped as a `.pkl` inside the package.

When to use: Drop-in replacement for Haiku/Sonnet when API access is unavailable or cost is constrained. Ideal for CI/CD pipelines and batch evaluation.

Performance: AUROC 0.747 ± 0.10 (5-fold CV, HotpotQA 200 chains, exp159). Cost: $0.000/chain.

```python
from llm_guard import MiniJudge

judge = MiniJudge()        # auto-loads bundled pkl
risk  = judge.score(question, steps, final_answer) # -> float [0,1]

# Score using alias (drop-in with AgentGuard)
risk  = judge.score_chain(question, steps, final_answer)

# Retrain on your own labeled data
judge.fit(labeled_chains) # each chain needs "correct" bool field
```

6.2 AgentGuard (Behavioral) — Production Baseline

What it is: Behavioral SC_OLD scoring using 12 structural features extracted from the ReAct chain. Fitted logistic-regression ensemble with no API dependency.

When to use: Default choice for production. $0 cost, 19 ms p50 latency. AUROC 0.817 within HotpotQA, 0.659–0.703 cross-domain.

Performance: AUROC 0.817 within-domain; alert precision 0.908 at FPR $\leq 10\%$ (exp92 conformal calibration).

```python
from llm_guard import AgentGuard

guard = AgentGuard() # no API key needed
result = guard.score_chain(question, steps, final_answer)

print(result.risk_score)     # float [0,1]
print(result.confidence_tier) # "HIGH" / "MEDIUM" / "LOW"
print(result.needs_alert)    # True when risk >= 0.70
print(result.failure_mode)   # "retrieval_fail" / None / ...
print(result.behavioral_score) # SC_OLD score before any blending
```

6.3 AgentGuard + P(True) — Cross-Domain Scoring

What it is: Behavioral SC_OLD blended with a P(True) signal from Haiku. P(True) prompts the model to rate answer correctness 1–5, providing a domain-agnostic uncertainty signal.

When to use: Cross-domain deployment where the target domain differs from the training distribution. Adds ~$0.0003/chain.

Performance: AUROC 0.78 cross-domain (best ensemble, exp120).

```python
from llm_guard import AgentGuard

guard = AgentGuard(api_key="sk-ant-...")
result = guard.score_with_ptrue(question, steps, final_answer,
                                ptrue_weight=0.9)

# Or use the adaptive bandit to tune ptrue_weight automatically
guard.enable_ptrue_bandit()
result = guard.score_with_ptrue(question, steps, final_answer)
```

6.4 AgentGuard + Sonnet Judge — Highest Precision

What it is: Full J5 ensemble: SC_OLD ×1 + Sonnet judge ×3. The Sonnet judge produces GOOD/BORDERLINE/POOR chain-level verdicts.

When to use: Compliance-sensitive pipelines, high-stakes answers, or when precision matters more than cost.

Performance: AUROC 0.777 within-domain, 0.741 cross-domain (exp89).

```python
from llm_guard import AgentGuard

guard = AgentGuard(api_key="sk-ant-...", use_judge=True)
result = guard.score_chain(question, steps, final_answer)

print(result.judge_label) # "GOOD" / "BORDERLINE" / "POOR"
print(result.risk_score) # J5 = SC_OLD*1 + judge*3 (normalised)
```

6.5 stream_guard — Mid-Chain Abort

What it is: Real-time abort decision evaluated at step 2 of the agent loop. Combines SC_OLD prefix scores with an optional Haiku mid-chain judge call.

When to use: Long-running chains where early abort saves 3–5 LLM calls. Particularly effective when empty observations appear at step 1–2.

Performance: Two-stage OR strategy achieves Recall 0.416, Precision 0.740 (vs J5-only: Recall 0.595, Precision 0.908). Use when recall matters.

```python
from llm_guard import AgentGuard

guard = AgentGuard()  # or AgentGuard(api_key=...) for Haiku judge

# Call after step 2 completes
sg = guard.stream_guard(question, steps_so_far, abort_threshold=0.65)

if sg.abort:
    print(f"Abort at step {sg.step_index}: risk={sg.risk_at_step:.3f}")
    # sg.rewritten_queries contains 3 diverse reformulations (if api_key set)
    next_query = sg.rewritten_queries[0] if sg.rewritten_queries else question
```

6.6 A2ATrustObject — Multi-Agent Trust

What it is: Structured confidence envelope passed between agents. HMAC-SHA256 signed over 8 canonical fields (exp114). Google A2A compatible.

When to use: Multi-agent orchestration. Agent A emits a trust object; Agent B conditions its strategy on the confidence tier and failure mode.

Performance: Round-trip pass 200/200 (100%); tamper detection 200/200 (100%); sign latency 0.034 ms; wire size 458–539 bytes.

```python
from llm_guard import AgentGuard, A2ATrustObject

guard = AgentGuard()
trust = guard.generate_trust_object(question, steps, final_answer)

# Agent A: sign and transmit
trust.sign("fleet-secret-2026")
payload = trust.to_dict() # JSON-serialisable dict

# Agent B: deserialise and verify
trust = A2ATrustObject.from_dict(payload)
assert trust.verify("fleet-secret-2026") # raises if tampered

if trust.should_rewrite:
    # Use QueryRewriter before running Agent B
    pass
```

6.7 QuickCalibrator — Domain Calibration

What it is: One-call domain-adaptive isotonic calibration. Wraps `calibrate_isotonic()` into a 20-chain setup flow.

When to use: Cross-domain deployment where default thresholds give too many false positives or misses. Needs 20+ labeled chains from the target domain.

Performance: Isotonic calibration improves threshold precision without changing AUROC (monotone transform). Particularly effective for domains with different base rates of failure.

```python
from llm_guard import QuickCalibrator

cal = QuickCalibrator(min_chains=20)
cal.fit(labeled_chains, domain="my_customer_support_bot")
# labeled_chains: each dict has "question", "steps",
#                "final_answer", "correct" (bool)

risk = cal.score(question, steps, final_answer)
```

```
# Returns calibrated risk in [0,1]
```

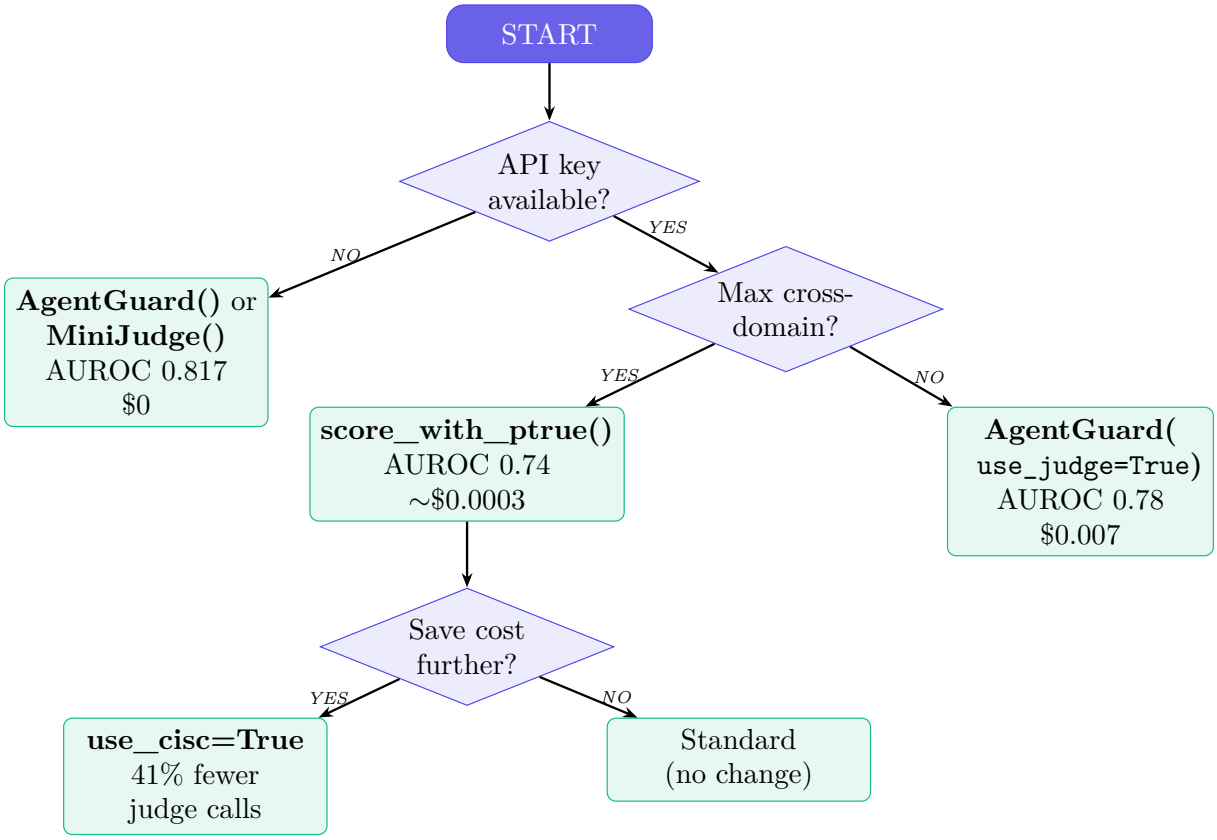
6.8 Platform Integrations

`llm-guard-kit` ships 2-line drop-in callbacks for major frameworks:

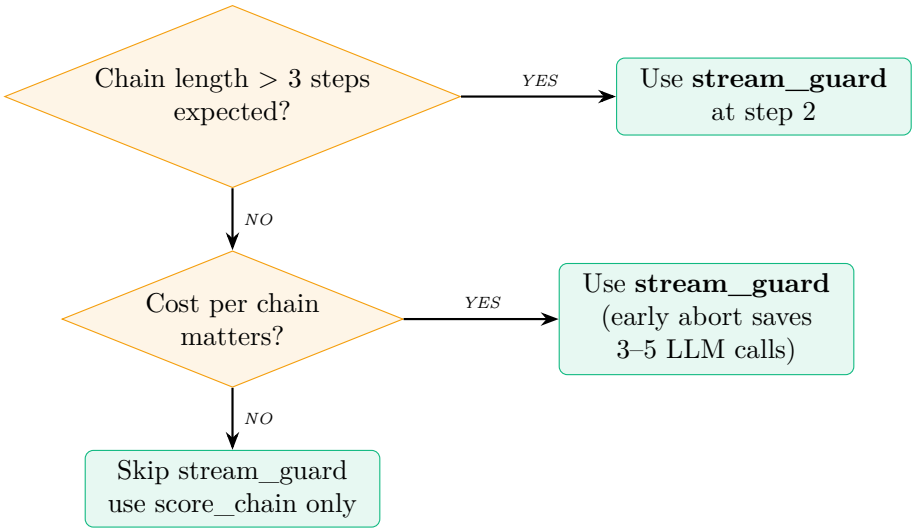
```
# LangChain
from llm_guard import AgentGuardCallback
from langchain.agents import AgentExecutor

agent = AgentExecutor(agent=..., tools=...,
                      callbacks=[AgentGuardCallback(api_key="sk-ant-...")])

# LlamaIndex
from llm_guard import AgentGuardEventHandler
from llama_index.core import Settings
Settings.callback_manager.add_handler(AgentGuardEventHandler())

# CrewAI
from llm_guard import AgentGuardCrewCallback
crew = Crew(agents=[...], tasks=[...],
            callbacks=[AgentGuardCrewCallback()])
```

7 Decision Guide

Use the flowchart below to select the right configuration for your use case.



When to use `stream_guard`:



8 Performance Benchmarks

All AUROC figures are from held-out evaluation sets; no in-sample figures are reported. Confidence intervals are computed via bootstrap (1000 resamples).

Method	Dataset	n	AUROC	95% CI	Cost/chain
SC_OLD behavioral	HotpotQA (within)	200	0.817	[0.74, 0.89]	\$0
MiniJudge (LogReg+SC_OLD)	HotpotQA 5-fold CV	200	0.747 ± 0.10	—	\$0
SC_OLD	TriviaQA (cross)	1000	0.659	[0.614, 0.705]	\$0
SC_OLD	2WikiMultiHop (cross)	200	0.703	[0.628, 0.775]	\$0
SC_OLD	NQ open-domain	200	0.524	[0.447, 0.609]	\$0
+P(True) Haiku	TriviaQA	37	0.720	—	~\$0.0003
+P(True) blend $\alpha = 0.25$	TriviaQA	37	0.737	—	~\$0.0003
SC_OLD+Sonnet (J5)	HotpotQA	200	0.777	—	~\$0.007
SC_OLD+Sonnet (J5)	TriviaQA	200	0.741	—	~\$0.007
Alert precision @ 0.70	HotpotQA	200	—	P=0.908	\$0
CISC cost reduction	HotpotQA	200	identical	—	−41%

Table 3: Full benchmark table. All results from held-out evaluation only. NQ open-domain AUROC 0.524 reflects the harder factoid domain gap; behavioral features are format-specific and degrade on truly unseen distributions.

9 Integration Patterns

9.1 Plain Python (simplest integration)

```python
from llm_guard import AgentGuard

guard = AgentGuard() # $0, no API key needed

# After your agent finishes
result = guard.score_chain(question, steps, final_answer)
if result.needs_alert:
    print(f"WARNING: risk={result.risk_score:.3f}, "
          f"mode={result.failure_mode}")
```

9.2 Langfuse (trace + score)

```python
from langfuse import Langfuse
from llm_guard import AgentGuard

langfuse = Langfuse()
guard    = AgentGuard()

def run_and_score(question, run_agent):
    with langfuse.trace(name="agent_run") as trace:
        steps, final_answer = run_agent(question)
        result = guard.score_chain(question, steps, final_answer)
        trace.score(
            name="llm_guard_risk",
            value=result.risk_score,
            comment=f"tier={result.confidence_tier}, "
                    f"mode={result.failure_mode}",
        )
    return final_answer, result
```

9.3 LangSmith (evaluator protocol)

```python
from langsmith.evaluation import EvaluationResult
from llm_guard import AgentGuard

guard = AgentGuard()

def llm_guard_evaluator(run, example):
    question    = example.inputs["question"]
    steps       = run.outputs.get("steps", [])
    final_answer = run.outputs.get("final_answer", "")
    result      = guard.score_chain(question, steps, final_answer)
    return EvaluationResult(
        key="llm_guard_risk",
        score=1.0 - result.risk_score, # higher = better
        comment=f"tier={result.confidence_tier}",
    )

# Then: client.evaluate(dataset_name, evaluators=[llm_guard_evaluator])
```

9.4 Prometheus (metrics + Grafana)

```python
from prometheus_client import Histogram, Counter, start_http_server
from llm_guard import AgentGuard

RISK_HIST = Histogram("llm_guard_risk_score", "Risk score distribution",
                      buckets=[0.1,0.2,0.3,0.4,0.5,0.6,0.7,0.8,0.9,1.0])
ALERT_CTR = Counter("llm_guard_alerts_total", "Number of alert-level chains")

guard = AgentGuard(on_alert=lambda r: ALERT_CTR.inc())

def score_and_observe(question, steps, final_answer):
    result = guard.score_chain(question, steps, final_answer)
    RISK_HIST.observe(result.risk_score)
    return result
```

```

start_http_server(8000) # Prometheus scrapes /metrics
```

9.5 Datadog (DogStatsD)

```python
from datadog import DogStatsd
from llm_guard import AgentGuard

statsd = DogStatsd()
guard  = AgentGuard()

def score_and_ship(question, steps, final_answer):
    result = guard.score_chain(question, steps, final_answer)
    statsd.gauge("llm_guard.risk_score", result.risk_score,
                 tags=[f"tier:{result.confidence_tier}",
                       f"mode:{result.failure_mode or 'none'}"])
    if result.needs_alert:
        statsd.increment("llm_guard.alerts",
                         tags=[f"mode:{result.failure_mode or 'none'}"])
    return result
```

10 API Quick Reference

10.1 Public Methods

Class	Method	Parameters	Returns
`MiniJudge`	`score()`	question, steps, final_answer	float [0,1]
`MiniJudge`	`score_chain()`	question, steps, final_answer	float [0,1]
`MiniJudge`	`fit()`	chains, n_splits=5	MiniJudge
`AgentGuard`	`score_chain()`	question, steps, final_answer	ChainTrustResult
`AgentGuard`	`score_with_ptrue()`	question, steps, final_answer	ChainTrustResult
`AgentGuard`	`generate_trust_object()`	question, steps, final_answer	A2ATrustObject
`AgentGuard`	`stream_guard()`	question, steps_so_far, abort_threshold	StreamGuardResult
`AgentGuard`	`activate_adapter()`	failure_mode	AdapterResult
`AgentGuard`	`route_to_mesh()`	question, trust, agent_answers	MeshResult
`AgentGuard`	`calibrate_isotonic()`	scores, labels	AgentGuard
`A2ATrustObject`	`sign()`	secret	A2ATrustObject
`A2ATrustObject`	`verify()`	secret	bool
`A2ATrustObject`	`to_dict()`	—	Dict
`QuickCalibrator`	`fit()`	chains, domain	QuickCalibrator
`QuickCalibrator`	`score()`	question, steps, final_answer	float
`probe_ensemble_blend()`	(module fn)	probe_score, ptrue_score, alpha=0.25	float

Table 4: All public API methods. `steps` is always a list of ReAct step dicts with keys: `thought`, `action_type`, `action_args`, `observation`.

10.2 ChainTrustResult Fields

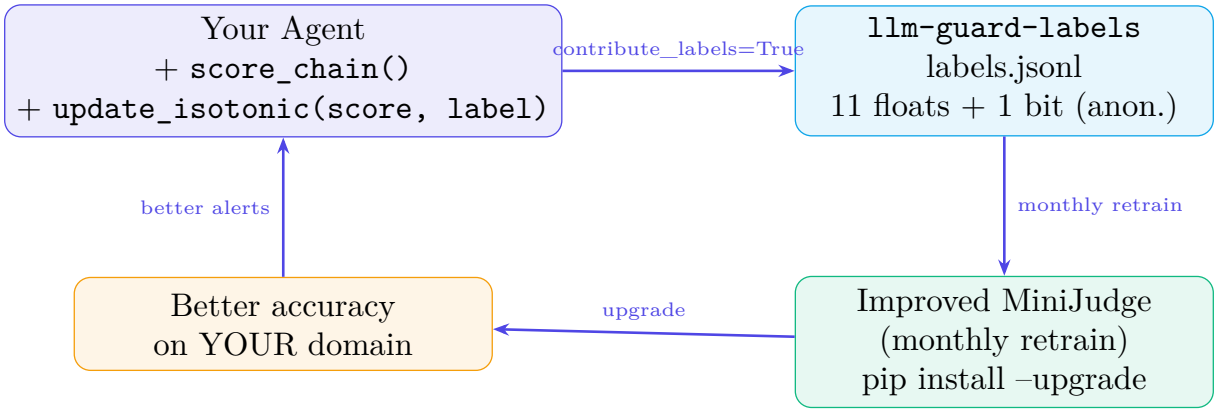
11 Telemetry Flywheel

11.1 Architecture

The telemetry flywheel allows users to contribute anonymised labels that improve the shared MiniJudge model over time.

Field	Type	Description
`risk_score`	float [0,1]	Composite risk (higher = more likely wrong)
`confidence_tier`	str	`"HIGH"` / `"MEDIUM"` / `"LOW"`
`needs_alert`	bool	True when risk_score ≥ alert_threshold (0.70)
`failure_mode`	str|None	`"retrieval_fail"` / `"repeated_query"` / `"long_chain"` / `"empty_answer"` / None
`judge_label`	str|None	`"GOOD"` / `"BORDERLINE"` / `"POOR"` (requires `api_key` + `use_judge=True`)
`behavioral_score`	float	SC_OLD score alone
`behavioral_components`	Dict	Individual SC_OLD feature scores (sc1–sc12)
`latency_ms`	float	Scoring latency in milliseconds
`step_count`	int	Number of Search steps in the chain

Table 5: Fields of `ChainTrustResult` returned by `score_chain()`.



11.2 Privacy Guarantee

Only the following data is transmitted when `contribute_labels=True`:

- **11 float features** (SC_OLD structural scores: loop rate, empty-observation rate, step count, etc.)
- **1 correctness bit** (0 = correct, 1 = wrong)

No question text, no answer text, no user data, no PII is transmitted. The 11 features are computed locally and contain no recoverable content. Label contribution is **opt-in only** and disabled by default.

```python
from llm_guard import AgentGuard

guard = AgentGuard()
result = guard.score_chain(question, steps, final_answer)

# After you know the ground truth:
guard.update_isotonic(result.behavioral_score, label=1)
# label=0 means correct, label=1 means wrong

# Opt-in to anonymised label contribution (default: False)
guard.contribute_label(result, correct=False,
                       github_pat=None) # set PAT to contribute
```

Quick Start

```
pip install llm-guard-kit

from llm_guard import AgentGuard

guard = AgentGuard() # zero cost, no API key

# Score a completed agent chain
result = guard.score_chain(
    question    = "What year was the Eiffel Tower built?",
    steps       = [
        {"thought": "Searching...", "action_type": "Search",
         "action_args": "Eiffel Tower year",
         "observation": "Built 1887-1889, completed for 1889 World Fair."},
        {"thought": "Got it.", "action_type": "Finish",
         "action_args": "1889", "observation": ""},
    ],
    final_answer = "1889",
)

print(f"Risk: {result.risk_score:.3f} Tier: {result.confidence_tier}")
# >> Risk: 0.133 Tier: HIGH

if result.needs_alert:
    print("ALERT: possible wrong answer")
```

PyPI: https://pypi.org/project/llm-guard-kit/ • **GitHub:** https://github.com/Avighan/llm-guard-kit